

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

**Лабораторная работа №3 по курсу
«Операционные системы»**

**Тема работы
“Межпроцессорное взаимодействие через memory-mapped files”**

Студент: Салихов Руслан Ринатович
Группа: М8О-203Б-23
Вариант: 19

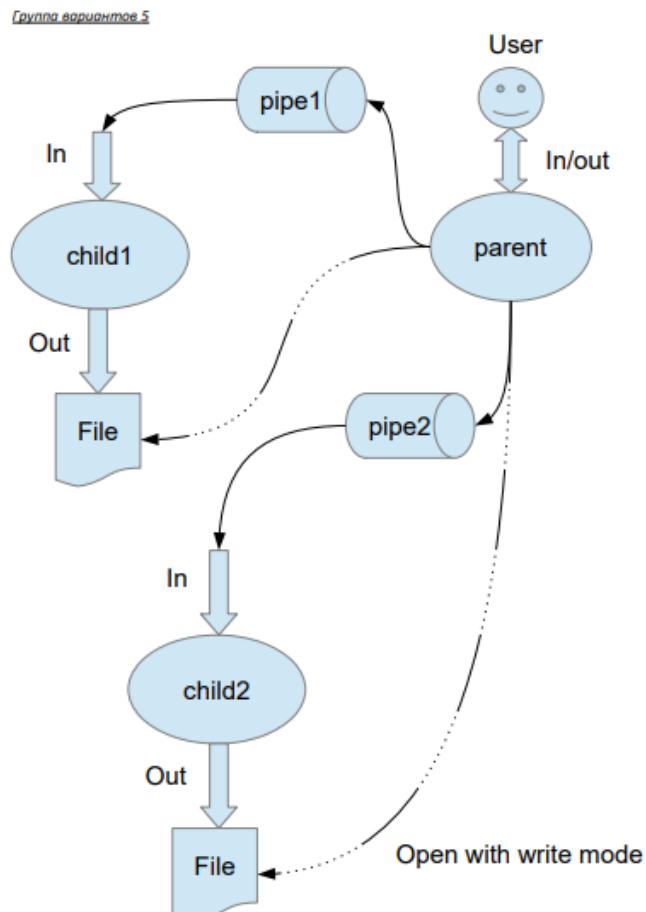
Преподаватель: Миронов Евгений Сергеевич

Оценка: _____
Дата: _____
Подпись: _____

Москва, 2024

Постановка задачи

Задача: реализовать программу, в которой родительский процесс создает два дочерних процесса. Межпроцессорное взаимодействие осуществляется посредством отображаемых файлов (memory-mapped files).



Вариант 19) Правило фильтрации: с вероятностью 80% строки отправляются в pipe1, иначе в pipe2.

Дочерние процессы удаляют все гласные из строк.

Общие сведения

Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в первый процесс или во второй процесс в зависимости от правила фильтрации. Процесс child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод.

Описание программы:

Реализовано взаимодействие между процессами с использованием разделяемой памяти и именованных семафоров для синхронизации. Родительский процесс создает разделяемую память и инициализирует необходимые семафоры, после чего ожидает ввода от пользователя двух имен файлов, в которые дочерние процессы будут записывать результаты обработки.

Родительский процесс передает введенные строки в разделяемую память, после чего, в зависимости от длины строки, передает управление одному из дочерних процессов. Дочерние процессы принимают строки, выполняют обработку, удаляя из них гласные буквы, а затем записывают полученный результат в соответствующий файл.

После завершения обработки родительский процесс передает дочерним процессам сигнал завершения работы и очищает созданные ресурсы, включая разделяемую память и семафоры. В ходе работы программы демонстрируется эффективное использование системных вызовов для обмена данными между процессами и управления ими.

Код программы

include/functions.hpp

```
#pragma once
#include <string>
#include <semaphore.h>

// Удаление гласных
std::string removeVowels(const std::string& input);

// Создание или открытие разделяемой памяти
int CreateShm(const char* name);
```

```

// Отображаемый файл (memory-mapped)
char* MapSharedMemory(int size, int fd);

// Создание/открытие именованного семафора
sem_t* CreateSemaphore(const char* name, int value);

// Бросаем ошибку при -1
void ErrorChecking(int result, const char* msg);

// --- Константы ---
constexpr int MMAP_SIZE = 4096; // размер области памяти (4 КБ)

// Имена для памяти (по одному на каждого ребёнка)
constexpr const char* SHM_NAME1 = "/myshm1";
constexpr const char* SHM_NAME2 = "/myshm2";

// Имена для семафоров (по 2 на каждого ребёнка: родитель->ребёнок, ребёнок->родитель)
constexpr const char* SEM_PARENT1 = "/sem_parent1";
constexpr const char* SEM_CHILD1 = "/sem_child1";

constexpr const char* SEM_PARENT2 = "/sem_parent2";
constexpr const char* SEM_CHILD2 = "/sem_child2";

```

include/parent.hpp

```

#pragma once

#include <istream>

void ParentProcess(const char* pathToChild1, const char* pathToChild2, std::istream&
streamIn);

```

src/child1.cpp

```

#include "functions.hpp"

#include <iostream>

#include <string>

#include <unistd.h>

#include <semaphore.h>

#include <sys/mman.h>

```

```

#include <fcntl.h>

#include <cstdlib>

int main(int argc, char* argv[]) {
    // Ожидаем 3 аргумента:
    // argv[1] - имя семафора родителя -> child1
    // argv[2] - имя семафора child1 -> родитель
    // argv[3] - имя shm
    if (argc < 4) {
        std::cerr << "Usage: child1 <semParent> <semChild> <shmName>\n";
        exit(EXIT_FAILURE);
    }

    const char* semParentName = argv[1];
    const char* semChildName = argv[2];
    const char* shmName = argv[3];

    sem_t* semParent = sem_open(semParentName, 0);
    sem_t* semChild = sem_open(semChildName, 0);
    if (semParent == SEM_FAILED || semChild == SEM_FAILED) {
        std::cerr << "child1: sem_open failed\n";
        exit(EXIT_FAILURE);
    }

    int shmFd = shm_open(shmName, O_RDWR, 0666);
    if (shmFd == -1) {
        std::cerr << "child1: shm_open failed\n";
        exit(EXIT_FAILURE);
    }

    char* addr = MapSharedMemory(MMAP_SIZE, shmFd);

```

```

while (true) {
    sem_wait(semParent); // ждём "разрешения" от родителя

    if (addr[0] == '\0') { // пустая строка -> конец
        break;
    }

    std::string input(addr);
    std::string output = removeVowels(input);
    std::cout << output << std::endl; // на stdout, перенаправленный в нужный файл
    sem_post(semChild); // сообщаем родителю, что обработали
}

munmap(addr, MMAP_SIZE);
close(shmFd);
sem_close(semParent);
sem_close(semChild);

return 0;
}

```

src/child2.cpp

```

#include "functions.hpp"
#include <iostream>
#include <string>
#include <unistd.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <cstdlib>

```

```

int main(int argc, char* argv[]) {
    if (argc < 4) {
        std::cerr << "Usage: child2 <semParent> <semChild> <shmName>\n";
        exit(EXIT_FAILURE);
    }

    const char* semParentName = argv[1];
    const char* semChildName = argv[2];
    const char* shmName = argv[3];

    sem_t* semParent = sem_open(semParentName, 0);
    sem_t* semChild = sem_open(semChildName, 0);
    if (semParent == SEM_FAILED || semChild == SEM_FAILED) {
        std::cerr << "child2: sem_open failed\n";
        exit(EXIT_FAILURE);
    }

    int shmFd = shm_open(shmName, O_RDWR, 0666);
    if (shmFd == -1) {
        std::cerr << "child2: shm_open failed\n";
        exit(EXIT_FAILURE);
    }

    char* addr = MapSharedMemory(MMAP_SIZE, shmFd);

    while (true) {
        sem_wait(semParent);
        if (addr[0] == '\0') {
            break;
        }
        std::string input(addr);
    }
}

```

```

        std::string output = removeVowels(input);

        std::cout << output << std::endl;

        sem_post(semChild);
    }

    munmap(addr, MMAP_SIZE);
    close(shmFd);
    sem_close(semParent);
    sem_close(semChild);

    return 0;
}

```

src/functions.cpp

```

#include "functions.hpp"
#include <iostream>
#include <algorithm>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <cstring>
#include <cerrno>
#include <cstdlib>

//удаляю гласные
std::string removeVowels(const std::string& input) {
    std::string result;
    result.reserve(input.size()); // выделяет память для хранения элементов размера input.size
    for (char c : input) {
        switch(c) {

```



```

        case 'a': case 'e': case 'i': case 'o': case 'u':
        case 'A': case 'E': case 'I': case 'O': case 'U':
            break; // пропускаю гласную
        default:
            result.push_back(c);
    }
}
return result;
}

// создаю или открываю shm (shared_memory)
int CreateShm(const char* name) {
    int fd = shm_open(name, O_CREAT | O_RDWR, 0666); // открываю именованный
    объект

        // имя для памяти // флаги // права доступа (rw для всех)
    if (fd == -1) {
        std::cerr << "shm_open error for " << name << ": " << strerror(errno) << std::endl;
        exit(EXIT_FAILURE);
    }

    //устанавливаем размер
    //MMAP_SIZE = 4096;

    if (ftruncate(fd, MMAP_SIZE) == -1) { //ftruncate(fd, MMAP_SIZE): устанавливаем
    размер

        // объекта fd(который открыли)

        std::cerr << "ftruncate error for " << name << ": " << strerror(errno) << std::endl;
        exit(EXIT_FAILURE);
    }

    return fd;
}

//функция является обёрткой над системным вызовом mmap, которая
//создаёт указатель на разделяемый файл

```

```

char* MapSharedMemory(int size, int fd) {
    void* addr = mmap(nullptr, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    // отображает объект в адресное пространство текущего процесса
    // mmap(nullptr, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0)
    // mmap(адрес отображения, кол-во байт, режим доступа, видимость изменений,
открытый объект)
    if (addr == MAP_FAILED) {
        std::cerr << "mmap failed: " << strerror(errno) << std::endl;
        exit(EXIT_FAILURE);
    }
    return static_cast<char*>(addr);
}

// семафоры — это примитив синхронизации, помогающий предотвратить проблемы,
// такие как (race conditions) и взаимные блокировки (deadlocks),
// которые могут возникнуть при конкурентном доступе к ресурсам.
sem_t* CreateSemaphore(const char* name, int value) {
    //      sem_open(имя семафора, флаг, rw для всех, начальное значение семафора)
    sem_t* sem = sem_open(name, O_CREAT, 0666, value);

    if (sem == SEM_FAILED) {
        std::cerr << "sem_open error for " << name << ": " << strerror(errno) << std::endl;
        exit(EXIT_FAILURE);
    }
    return sem;
}

// ВЫЗОВ ОШИБКИ
void ErrorChecking(int result, const char* msg) {
    if (result == -1) {
        perror(msg);
        exit(EXIT_FAILURE);
    }
}

```

```
}  
  
}
```

src/parent.cpp

```
#include <cstring>    // memset, memcpy  
#include <sys/mman.h> // shm_unlink, mmap, munmap  
#include <sys/stat.h> // ftruncate, ...  
#include <fcntl.h>    // shm_open, open, O_*...  
#include <unistd.h>   // fork, close, dup2  
#include <sys/wait.h> // waitpid  
#include <iostream>  
#include <string>  
#include <cstdlib>    // rand, srand, exit  
#include <ctime>      // time  
#include "parent.hpp"  
#include "functions.hpp"  
  
#define _XOPEN_SOURCE 700  
  
void ParentProcess(const char* pathToChild1, const char* pathToChild2, std::istream&  
streamIn) {  
    // путь к исп. файлу child1 //путь к исп. файлу child2 //поток ввода  
  
    std::string file1, file2;  
  
    //читаю имя файла 1  
    std::cout << "Введите имя файла для child1: ";  
    if (!std::getline(streamIn, file1)) {  
        std::cerr << "Ошибка чтения имени файла для child1\n";  
        exit(EXIT_FAILURE);  
    }  
}
```

```

//читаю имя файла 2
std::cout << "Введите имя файла для child2: ";
if (!std::getline(streamIn, file2)) {
    std::cerr << "Ошибка чтения имени файла для child2\n";
    exit(EXIT_FAILURE);
}

//открываю эти файлы на запись, куда в дальнейшем будет направлен stdout дочерних
процессов
int fd1 = open(file1.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0666); //открыть на
запись, создать при отсутствии,

//очистить файл, если он уже есть.
if (fd1 == -1) {
    perror("open file1");
    exit(EXIT_FAILURE);
}

int fd2 = open(file2.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0666); //открыть на
запись, создать при отсутствии,

//очистить файл, если он уже есть.
if (fd2 == -1) {
    perror("open file2");
    close(fd1);
    exit(EXIT_FAILURE);
}

//SHM_NAME1, SHM_NAME2 имена для файлов прописанные в functions.hpp
shm_unlink(SHM_NAME1); // предварительно очищаю объекты перед новым вызовом
shm_unlink(SHM_NAME2);
int shmFd1 = CreateShm(SHM_NAME1);
int shmFd2 = CreateShm(SHM_NAME2);

```

```

// удаление семафора, тоже как бы очищая память от предыдущего запуска
sem_unlink(SEM_PARENT1);
sem_unlink(SEM_CHILD1);
sem_unlink(SEM_PARENT2);
sem_unlink(SEM_CHILD2);

sem_t* semParent1 = CreateSemaphore(SEM_PARENT1, 0);
sem_t* semChild1 = CreateSemaphore(SEM_CHILD1, 0);
sem_t* semParent2 = CreateSemaphore(SEM_PARENT2, 0);
sem_t* semChild2 = CreateSemaphore(SEM_CHILD2, 0);

// дочерний процесс #1
pid_t pid1 = fork();
ErrorChecking(pid1, "fork child1");
if (pid1 == 0) {
    // перенаправляю stdout в fd1, т.е. работает дочерний процесс 1
    dup2(fd1, STDOUT_FILENO);
    close(fd1);
    close(fd2);

    //Выполняю execl, чтобы запустить внешний исполняемый файл child1
    execl(pathToChild1, pathToChild1, SEM_PARENT1, SEM_CHILD1, SHM_NAME1,
    nullptr);
    perror("exec child1");
    exit(EXIT_FAILURE);
}

// дочерний процесс #2
pid_t pid2 = fork();
ErrorChecking(pid2, "fork child2");
if (pid2 == 0) {
    dup2(fd2, STDOUT_FILENO);
    close(fd2);

```

```

    close(fd1);

    execl(pathToChild2, pathToChild2, SEM_PARENT2, SEM_CHILD2, SHM_NAME2,
    nullptr);

    perror("exec child2");
    exit(EXIT_FAILURE);
}

// если мы тут, значит это всё ещё родитель.
close(fd1);
close(fd2);

// генератор псевдослучайных чисел
std::srand((unsigned)std::time(nullptr));

// Цикл чтения строк из streamIn
std::string line;
while (true) {
    std::cout << "Введите строку (EOF для выхода): ";
    if (!std::getline(streamIn, line)) {
        //Если не можем прочитать строку -> EOF
        break;
    }
    if (line.empty()) {
        break;
    }
    //80% в child1, 20% в child2
    bool toChild1 = (std::rand() % 10 < 8);
    if (toChild1) {
        // отправляю строку в child1
        char* addr = MapSharedMemory(MMAP_SIZE, shmFd1); // отобразить shm1 в
        память
        memset(addr, 0, MMAP_SIZE); // очистить
    }
}

```

```

        memcpy(addr, line.c_str(), line.size());    // записать строку

        // даю сигнал child1, что есть данные

        sem_post(semParent1);    // sem_post(sem_t *sem); увеличивает значение
семафора

        // ожидаю что обратано

        sem_wait(semChild1);

        // высвободил память

        munmap(addr, MMAP_SIZE);

    } else {

        //то же самое в child2

        char* addr = MapSharedMemory(MMAP_SIZE, shmFd2);

        memset(addr, 0, MMAP_SIZE);

        memcpy(addr, line.c_str(), line.size());

        sem_post(semParent2);

        sem_wait(semChild2);

        munmap(addr, MMAP_SIZE);

    }

}

//сигналим детям завершение чтобы они вышли из своих циклов чтения
{

    // child1

    char* addr1 = MapSharedMemory(MMAP_SIZE, shmFd1);

    addr1[0] = '\0'; // признак окончания

    sem_post(semParent1); //указываю ребёнку что строка пуста (увеличивает значение
семафора)

    munmap(addr1, MMAP_SIZE);

    // child2

    char* addr2 = MapSharedMemory(MMAP_SIZE, shmFd2);

    addr2[0] = '\0';

    sem_post(semParent2);

    munmap(addr2, MMAP_SIZE);

```

```

}

// жду заверение чаилдов
waitpid(pid1, nullptr, 0);
waitpid(pid2, nullptr, 0);

//закрываю всё
close(shmFd1);
close(shmFd2);
shm_unlink(SHM_NAME1);
shm_unlink(SHM_NAME2);

sem_close(semParent1);
sem_close(semChild1);
sem_close(semParent2);
sem_close(semChild2);
sem_unlink(SEM_PARENT1);
sem_unlink(SEM_CHILD1);
sem_unlink(SEM_PARENT2);
sem_unlink(SEM_CHILD2);
}

```

tests/test.cpp

```

#include <gtest/gtest.h>
#include <fstream>
#include <sstream>
#include <cstdio>
#include <parent.hpp>

// функция для чтения файла в строку
static std::string readFileContent(const std::string &filename) {

```



```

std::ifstream file(filename);

std::stringstream buffer;

buffer << file.rdbuf();

return buffer.str();
}

TEST(MmapLabTest, BasicCheck) {
    // пути к исполняемым файлам child1/child2 (из переменных окружения)
    const char* pathToChild1 = std::getenv("CHILD1_PATH");
    const char* pathToChild2 = std::getenv("CHILD2_PATH");
    ASSERT_NE(pathToChild1, nullptr) << "CHILD1_PATH is not set";
    ASSERT_NE(pathToChild2, nullptr) << "CHILD2_PATH is not set";

    std::string file1 = "test_out1.txt"; // временные файлы
    std::string file2 = "test_out2.txt";
    std::remove(file1.c_str());
    std::remove(file2.c_str());

    //вход. данные для parent process
    std::istringstream input_stream(
        file1 + "\n" +           // сразу передаю имя файлов
        file2 + "\n" +
        "Hello\n" +
        "World\n" +
        "AEIOU\n" +
        "xyz\n"
        "\n" // завершил ввод
    );

    // запускаю родителя, передав поток
    ParentProcess(pathToChild1, pathToChild2, input_stream);
}

```

```

// читаю что оказалось в файлах
std::string c1 = readFileContent(file1);
std::string c2 = readFileContent(file2);
std::string total = c1 + c2;

//проверка на удаление гласных
EXPECT_TRUE(total.find("Hll") != std::string::npos);
EXPECT_TRUE(total.find("Wrld") != std::string::npos);

for (char v : std::string("AEIOUaeiou")) {
    EXPECT_EQ(total.find(v), std::string::npos);
}

EXPECT_TRUE(total.find("xyz") != std::string::npos);
std::remove(file1.c_str());
std::remove(file2.c_str());
}

// если ввод пустлй (проверка)
TEST(MmapLabTest, EmptyInput) {
    const char* pathToChild1 = std::getenv("CHILD1_PATH");
    const char* pathToChild2 = std::getenv("CHILD2_PATH");
    ASSERT_NE(pathToChild1, nullptr);
    ASSERT_NE(pathToChild2, nullptr);

    // Два временных файла
    std::string file1 = "test_out1.txt";
    std::string file2 = "test_out2.txt";

    std::remove(file1.c_str()); // c_str() - преобразование объекта std::string в указатель const char*
    std::remove(file2.c_str()); // тк remove принимает строку в си формате

    std::istringstream input_stream( // строковой поток ввода + eof

```

```

        file1 + "\n" +
        file2 + "\n"
    );

    ParentProcess(pathToChild1, pathToChild2, input_stream);

    // Проверяем, что оба файла пусты
    std::string c1 = readFileContent(file1);
    std::string c2 = readFileContent(file2);
    EXPECT_TRUE(c1.empty());
    EXPECT_TRUE(c2.empty());

    std::remove(file1.c_str());
    std::remove(file2.c_str());
}

int main(int argc, char** argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}

```

Пример вывода:

```

user@DESKTOP-
Q8EOOMG:/mnt/c/Users/user/Desktop/Labs/os/lr3/build$
./start_test

```

```
[=====] Running 2 tests from 1 test suite.
```

```
[-----] Global test environment set-up.
```

```
[-----] 2 tests from MmapLabTest
```

```
[ RUN      ] MmapLabTest.BasicCheck
```

Введите имя файла для child1: Введите имя файла для
 child2: Введите строку (EOF для выхода): Введите строку
 (EOF для выхода): Введите строку (EOF для выхода):

```
Введите строку (EOF для выхода): Введите строку (EOF для
выхода): [          OK ] MmapLabTest.BasicCheck (22 ms)
[ RUN          ] MmapLabTest.EmptyInput
Введите имя файла для child1: Введите имя файла для
child2: Введите строку (EOF для выхода): [          OK ]
MmapLabTest.EmptyInput (7 ms)
[-----] 2 tests from MmapLabTest (30 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (30 ms total)
[  PASSED  ] 2 tests.
```

Вывод

ходе выполнения данной лабораторной работы были изучены и освоены методы взаимодействия между процессами, включая их создание, управление и синхронизацию. Особый акцент был сделан на использовании сигналов для координации работы процессов и обеспечения их корректного взаимодействия.

Кроме того, была детально изучена работа с файлами, отображенными в память (memory-mapped files), что позволило углубить понимание эффективных методов обмена данными между процессами. Освоенные навыки значительно улучшили понимание механизмов межпроцессного взаимодействия и работы с системными ресурсами.